



Práctica Sprint 05 — Asistente de consultas del bootcamp

Práctica integradora del Sprint 5 (Prompt & Context Engineering).

Construirás un asistente que:

1. **Recibe consultas** de alumnos (nombre, email, mensaje).
 2. **Valida** los datos en Python.
 3. **Clasifica** el mensaje con Gemini → JSON.
 4. **Responde** en modo chat con FAQ filtrado, historial y métricas de tokens.
-

Orden recomendado

Sigue este orden **de arriba a abajo**. No saltes a `logic.py` hasta tener listas las funciones que llama.

Fase 1 — clasificación

- ☐ 1. `validators.py` → `validar_consulta()`
- ☐ 2. `prompts.py` → `build_clasificacion_prompt()` (solo esta función de prompts)
- ☐ 3. `logic.py` → `parsear_clasificacion()`
- ☐ 4. `logic.py` → `clasificar_consulta()`
- ☐ 5. Ejecuta `python main.py` → la demo 1 debe mostrar `[OK]` / `[ERROR]` (no `[PENDIENTE]`)

Fase 2 — chat con contexto

- ☐ 6. `context.py` → `seleccionar_faq()`
- ☐ 7. `prompts.py` → `build_perfil_block`, `build_faq_block`, `build_historial_block`, `build_chat_prompt`
- ☐ 8. `state.py` → `append_user`, `append_model`, `ultimos_n`
- ☐ 9. `logic.py` → `demo_seleccion_faq()` y `responder_chat()`
- ☐ 10. Ejecuta `python main.py` → demo 2 completa con métricas y memoria del nombre

Archivos que **no debes modificar**

main.py, config.py, gemini_auth.py, gemini_client.py, data/*.json

Cómo probar sin print

En logic.py y validators.py **no uses print**. Para ver resultados, ejecuta siempre:

```
python main.py
```

main.py ya imprime los resultados por ti con imprimir_resultado().

Helpers que ya existen (no los reescribas)

En logic.py ya están hechas respuesta_ok() y respuesta_error(). Úsalas así:

```
return respuesta_error("Consulta inválida", errores) # cuando validar falle
return respuesta_ok("Clasificación completada", obj) # cuando todo vaya bien
```

Glosario rápido

Término	Significado en esta práctica
dict	Diccionario Python: {"clave": "valor"}
raw	Texto crudo que devuelve Gemini (a veces es JSON en string)
whitelist	Lista cerrada de valores permitidos (CATEGORIAS, PRIORIDADES en config.py)
state	Dict en memoria que guarda perfil + historial del chat entre turnos
prompt	Texto completo que envías al modelo (rol + instrucciones + contexto + pregunta)
llamar_gemini_json()	Llama a Gemini y espera solo JSON (fase 1)
safe_generate_texto()	Llama a Gemini y espera texto libre + métricas (fase 2)
WINDOW	Cuántos mensajes recientes del chat se incluyen en el prompt (config.py)
NotImplementedError	Normal al inicio: significa que aún no implementaste esa función

Requisitos

- Python 3.10+
- Cuenta en [Google AI Studio](#) (GEMINI_API_KEY)

Entorno virtual

Linux / macOS (bash):

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # edita tu clave dentro de .env
python main.py
```

Windows (PowerShell):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env       # edita tu clave dentro de .env
python main.py
```

Si PowerShell bloquea la activación del venv, ejecuta una vez (como administrador o en tu usuario):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Sin `.env`, `gemini_auth.py` pedirá la clave con `getpass` al usar la API.

Estructura del proyecto

```
.
├── README.md           ← este enunciado
├── requirements.txt    ← dependencias (pip install -r)
├── .gitignore          ← excluye .env, .venv, etc.
├── .env.example        ← plantilla de API key (sí va al repo)
├── .env               ← tu clave real (lo creas tú; no en Git)
├── config.py          ← constantes (modelo, límites, categorías)
├── validators.py       ← validar_consulta()
├── prompts.py         ← build_*_prompt()
├── context.py         ← seleccionar_faq()
├── state.py           ← historial y perfil
├── logic.py           ← clasificar_consulta(), responder_chat()
├── gemini_auth.py     ← configuración de GEMINI_API_KEY
├── gemini_client.py   ← llamadas a Gemini y métricas
├── data/
│   ├── faq.json
│   └── consultas_ejemplo.json
└── main.py           ← demos (no redefinir reglas aquí)
```

Quién importa a quién

```
main → logic, state, context

logic → validators, prompts, gemini_client, context, config, state

context, validators, prompts → config (no importan logic ni main)
```

FASE 1 - Validación y clasificación de consultas

Objetivo

Validar consultas y clasificarlas en JSON **sin llamar al modelo** si el input es inválido.

Orden mental: **primero Python valida → solo si pasa, llamas a Gemini.**

Al terminar esta fase, `demo_clasificar_consultas()` en `main.py` debe imprimir `[OK]` o `[ERROR]` para cada fila de `data/consultas_ejemplo.json`.

Tarea 1 — `validators.py` → `validar_consulta(datos)`

Qué hace: comprueba que la consulta del alumno tiene formato correcto **antes** de gastar tokens en la API.

Entrada esperada (dict, como en `consultas_ejemplo.json`):

```
datos = {
    "nombre": "Ana García",
    "email": "ana.garcia@ejemplo.com",
    "mensaje": "No entiendo cuándo es la live review del Sprint 5.",
}
```

Salida esperada:

```
[ ] # lista vacía = todo OK

# o, si hay problemas:
[
    "Nombre inválido: no puede estar vacío.",
    "Email inválido: formato incorrecto.",
    "Mensaje demasiado corto (mínimo 10 caracteres).",
]
```

Reglas (constantes en `config.py`):

Campo	Regla
<code>nombre</code>	No vacío (tras quitar espacios)

Campo	Regla
email	No vacío y formato válido (PATRON_EMAIL)
mensaje	Entre MIN_CHARS_MENSAJE y MAX_CHARS_MENSAJE caracteres

Prueba rápida: la tercera consulta del JSON de ejemplo debe devolver **varios** errores a la vez.

Pseudocódigo de ayuda (adapta los mensajes exactos):

```
errores = []
nombre = str(datos.get("nombre", "")).strip()
if not nombre:
    errores.append("Nombre inválido: ...")

email = str(datos.get("email", "")).strip()
if not email:
    errores.append("Email inválido: ...")
elif PATRON_EMAIL.fullmatch(email) is None:
    errores.append("Email inválido: formato incorrecto.")

mensaje = str(datos.get("mensaje", "")).strip()
if len(mensaje) < MIN_CHARS_MENSAJE:
    errores.append(f"Mensaje demasiado corto ...")
elif len(mensaje) > MAX_CHARS_MENSAJE:
    errores.append(f"Mensaje demasiado largo ...")

return errores
```

Tarea 2 — `prompts.py` → `build_clasificacion_prompt(mensaje)`

Qué hace: monta el **texto completo** que enviarás a Gemini. Solo ensambla strings; **no llama a la API aquí**.

Entrada:

```
mensaje = "Mi GEMINI_API_KEY no funciona, ¿qué reviso?"
```

Salida: un `str` con tres bloques en este orden:

1. `ROLE_CLASIFICADOR` (quién es el modelo)
2. `TASK_CLASIFICAR` (qué debe devolver en JSON)
3. El mensaje del alumno

Ejemplo de estructura (no copies literal si tu redacción es equivalente):

```
Eres un analista de consultas del bootcamp...
```

Tarea: clasifica el mensaje...

Mensaje:

Mi GEMINI_API_KEY no funciona, ¿qué reviso?

Usa las constantes `ROLE_CLASIFICADOR` y `TASK_CLASIFICAR` que ya están en el archivo.

Tarea 3 — `logic.py` → `parsear_clasificacion(raw)`

Qué hace: convierte la **respuesta en texto** del modelo en un `dict` Python **seguro** (whitelist).

Entrada (`raw` = string que devuelve `llamar_gemini_json`):

```
raw = '{"category": "tecnico", "priority": "media", "summary": "Problema con API key"}'
```

Salida si todo es correcto:

```
{
    "category": "tecnico",
    "priority": "media",
    "summary": "Problema con API key",
}
```

Debe rechazar (lanzar `ValueError` con mensaje claro):

- JSON mal formado
 - Faltan claves `category`, `priority`, `summary`
 - `category` fuera de `CATEGORIAS` (`academico`, `tecnico`, `administrativo`, `otro`)
 - `priority` fuera de `PRIORIDADES` (`baja`, `media`, `alta`)
-

Tarea 4 — `logic.py` → `clasificar_consulta(datos)`

Qué hace: orquesta la fase 1. **No uses `print`**; devuelve siempre un dict con `status`, `mensaje` y `data`.

Flujo:

```
validar_consulta(datos)
├─ si hay errores → respuesta_error(...)
└─ si OK → build_clasificacion_prompt(mensaje)
    → llamar_gemini_json(prompt)    # import en gemini_client
    → parsear_clasificacion(raw)
    → respuesta_ok(..., obj)
```

Entrada:

```
datos = {
    "nombre": "Luis Pérez",
    "email": "luis@ejemplo.com",
    "mensaje": "Mi GEMINI_API_KEY no funciona en el notebook, ¿qué reviso?",
}
```

Salida si la validación falla:

```
{
    "status": "error",
    "mensaje": "Consulta inválida",
    "data": {"errores": ["...", "..."]},
}
```

Salida si todo va bien:

```
{
    "status": "ok",
    "mensaje": "Clasificación completada",
    "data": {
        "category": "tecnico",
        "priority": "media",
        "summary": "...",
    },
}
```

`main.py` ya tiene `imprimir_resultado()` para mostrar esto en consola; no dupliques esa lógica en `logic.py`.

Criterios de aceptación (fase 1)

- ☐ `python main.py` pasa verificación estructural.
- ☐ La consulta inválida de `consultas_ejemplo.json` devuelve `[ERROR]` con **varios** errores listados.
- ☐ Las consultas válidas devuelven `[OK]` con `category`, `priority`, `summary`.
- ☐ `category` solo usa: `academico`, `tecnico`, `administrativo`, `otro`.
- ☐ No hay `print` en `logic.py` ni `validators.py`.

Pista JSON esperado

```
{
    "category": "academico",
    "priority": "media",
}
```

```
"summary": "Duda sobre live review del Sprint 5"
}
```

FASE 2 - Chat con contexto del bootcamp

Objetivo

Responder en modo conversación usando **solo el contexto necesario**: una entrada del FAQ, el perfil del alumno y los últimos turnos del chat.

Al terminar, `demo_chat_con_contexto()` en `main.py` debe: (1) seleccionar FAQ, (2) mantener memoria del nombre en el tercer turno, (3) mostrar métricas de tokens.

Tarea 1 — `context.py` → `seleccionar_faq(faq, consulta, max_entradas=1)`

Qué hace: elige **1 entrada** de `faq.json` relevante para la pregunta. **No metas todo el FAQ en el prompt.**

Entrada:

```
consulta = "¿Qué es la live review del bootcamp?"
# faq = lista cargada con cargar_faq(...) # 6 entradas
```

Salida esperada:

```
[
  {
    "topic_id": "live_review",
    "keywords": ["live review", "práctica", ...],
    "question": "¿Qué es la live review?",
    "answer": "Son dos sesiones...",
  }
]
```

Lógica sugerida: sumar puntos si una `keyword` aparece en la consulta; bonus si `topic_id` aparece en el texto; quedarte con las de mayor puntuación (máx. `max_entradas`).

Si no hay coincidencias: devuelve lista vacía `[]` (`demo_seleccion_faq` convertirá eso en error).

Pseudocódigo de ayuda:

```
consulta_lower = consulta.lower()
puntuaciones = [] # lista de (puntos, entrada_faq)

for entrada in faq:
    puntos = 0
```



```
for keyword in entrada["keywords"]:
    if keyword.lower() in consulta_lower:
        puntos += 2
if entrada["topic_id"].lower() in consulta_lower:
    puntos += 3
if puntos > 0:
    puntuaciones.append((puntos, entrada))

# ordenar de mayor a menor puntuación y devolver las top max_entradas
```

Tarea 2 — `prompts.py` → bloques y `build_chat_prompt(...)`

Cuatro funciones que **solo devuelven texto** para ensamblar el prompt del chat.

`build_perfil_block(profile)`

Entrada:

```
profile = {
    "name": "Ana",
    "email": "ana@ejemplo.com",
    "language": "español",
    "level": "junior",
}
```

Salida: bloque delimitado con `--- PERFIL DEL ALUMNO --- ... --- FIN PERFIL ---`, o `""` si `profile` está vacío.

`build_faq_block(faq_entries)`

Entrada: lista con **una** entrada (la que devolvió `seleccionar_faq`), no el JSON completo.

Salida: bloque con pregunta y respuesta del FAQ, entre `--- FAQ BOOTCAMP ---` y `--- FIN FAQ ---`.

`build_historial_block(messages)`

Entrada:

```
messages = [
    {"role": "user", "text": "Me llamo Ana y estudio el Sprint 5."},
    {"role": "model", "text": "Encantado, Ana. ¿En qué puedo ayudarte?"},
]
```

Salida: líneas `user: ...` y `model: ...` entre delimitadores, o `""` si no hay mensajes.

`build_chat_prompt(pregunta, profile, faq_entries, recent_messages)`

Qué hace: rellena `PLANTILLA_CHAT` con los tres bloques anteriores + la pregunta actual.

Salida: un único `str` listo para `safe_generate_texto()` en `gemini_client.py`.

Tarea 3 — `state.py` → memoria de la sesión

El `state` es un dict que **persiste entre turnos** del chat (en memoria, no en disco).

Estructura (ya la crea `inicializar_estado`):

```
{
    "user_profile": {"name": "Ana", "email": "...", ...},
    "messages": [],          # {"role": "user"|"model", "text": "..."}
    "consultas_clasificadas": [],
}
```

Función	Qué hace
<code>append_user(state, texto)</code>	Añade <code>{"role": "user", "text": texto}</code> a <code>messages</code>
<code>append_model(state, texto)</code>	Añade <code>{"role": "model", "text": texto}</code> a <code>messages</code>
<code>ultimos_n(state, n)</code>	Devuelve los últimos <code>n</code> mensajes (usa <code>WINDOW</code> desde <code>config</code>)

Pseudocódigo de ayuda:

```
# append_user
state["messages"].append({"role": "user", "text": texto.strip()})

# append_model
state["messages"].append({"role": "model", "text": texto.strip()})

# ultimos_n — últimos n mensajes (o [] si n <= 0)
return state["messages"][-n:]
```

Importante: en `responder_chat`, guarda en el historial **después** de recibir respuesta correcta del modelo.

Tarea 4 — `logic.py` → `demo_seleccion_faq` y `responder_chat`

`demo_seleccion_faq(faq_path, consulta)`

Qué hace: prueba la selección de FAQ sin llamar al chat completo.

Salida OK:

```
{
    "status": "ok",
```

```
"mensaje": "Entrada FAQ seleccionada",
"data": {"topic_id": "live_review", "entry": {...}},
}
```

responder_chat(state, pregunta, faq_entries)

Entrada:

```
state = inicializar_estado({"name": "Ana", ...})
pregunta = "¿Cuándo son las clases en directo?"
faq_entries = seleccionar_faq(...) # 0 o 1 entradas
```

Flujo:

```
validar pregunta no vacía
→ build_chat_prompt(perfil + faq + ultimos_n(state, WINDOW) + pregunta)
→ safe_generate_texto(prompt)
→ append_user + append_model en state
→ respuesta_ok con texto y métricas
```

Salida OK:

```
{
  "status": "ok",
  "mensaje": "Respuesta generada",
  "data": {
    "respuesta": "Las live sessions suelen ser de 19:00 a 21:00...",
    "metricas": {
      "elapsed_ms": 850,
      "prompt_tokens": 420,
      "output_tokens": 65,
      "total_tokens": 485,
    },
  },
}
```

Pseudocódigo de ayuda:

```
if not pregunta.strip():
    return respuesta_error("Pregunta vacía", [...])

prompt = build_chat_prompt(
    pregunta=pregunta,
    profile=state["user_profile"],
    faq_entries=faq_entries,
```

```
        recent_messages=ultimos_n(state, WINDOW),
    )

    texto, metricas = safe_generate_texto(prompt)    # import desde gemini_client

    append_user(state, pregunta)
    append_model(state, texto)

    return respuesta_ok(MSG_CHAT_OK, {"respuesta": texto, "metricas": {...}})
```

Criterios de aceptación (chat)

- ☐ Pregunta sobre "live review" selecciona la entrada FAQ correcta (no las 6).
- ☐ Tras presentarse, el asistente responde **cómo se llama** el alumno en el tercer turno.
- ☐ Consola muestra **elapsed_ms** y tokens en la respuesta de chat.
- ☐ Si el prompt supera **MAX_TOKENS_INPUT**, devuelve error claro (sin llamar al modelo).

Turnos de prueba (**demo_chat_con_contexto** en **main.py**)

1. "Me llamo Ana y estudio el Sprint 5 del bootcamp."
2. "¿Cuándo son las clases en directo?"
3. "¿Cómo me llamo y qué sprint estoy estudiando?"

Qué ejecutar y qué ver en consola

Un solo comando ejecuta las tres demos en orden:

```
python main.py
```

Demo 0 — Verificación (siempre funciona)

```
0) Verificación de estructura (sin API)
   FAQ cargado: 6 entradas
   Consultas ejemplo: 4 registros
   Estructura OK. Completa los TODO del proyecto.
```

No necesitas API key para esta parte.

Demo 1 — Clasificación (fase 1)

Con los TODOs completados y **GEMINI_API_KEY** configurada:

```
1) Validación + clasificación JSON
```

```
--- Ana García ---
[OK] Clasificación completada
  → academico / media: Duda sobre live review...

--- Luis Pérez ---
[OK] Clasificación completada
  → tecnico / ...

--- (sin nombre) ---
[ERROR] Consulta inválida
  - Nombre inválido: no puede estar vacío.
  - Email inválido: formato incorrecto.
  - Mensaje demasiado corto (mínimo 10 caracteres).
```

Si aún no implementaste la fase 1 verás **[PENDIENTE – clasificación]** y la demo 2 puede seguir intentándose.

Demo 2 — Chat (fase 2)

```
2) FAQ filtrado + chat con memoria

Selección FAQ: ¿Qué es la live review del bootcamp?
[OK] Entrada FAQ seleccionada
  FAQ topic: live_review
  P: ¿Qué es la live review?...

--- Usuario: Me llamo Ana...
[OK] Respuesta generada
  ...
  métricas: 850 ms | in=420 out=65
```

En el **tercer turno**, la respuesta debe mencionar **Ana** y **Sprint 5** gracias al historial en **state**.

Errores frecuentes (y qué hacer)

Lo que ves	Qué significa	Qué hacer
[PENDIENTE – clasificación]	Falta implementar algún TODO de fase 1	Revisa el checklist «Empieza aquí» y el mensaje del error
NotImplementedError: Implementa ...	La función sigue con raise	Implementa esa función concreta
ModuleNotFoundError: google...	Dependencias no instaladas	Activa el venv y pip install -r requirements.txt
Error de API / 401 / clave inválida	GEMINI_API_KEY mal o vacía	Revisa .env o pega la clave cuando getpass la pida

Lo que ves	Qué significa	Qué hacer
[ERROR] Consulta inválida con lista de errores	Es correcto para la consulta mala del JSON	No es un bug: la validación funciona
Demo 1 OK pero demo 2 [PENDIENTE]	Fase 1 lista, fase 2 no	Sigue con el checklist desde el punto 6
El chat no recuerda el nombre en turno 3	Historial no se guarda o WINDOW muy pequeño	Revisa append_user / append_model y ultimos_n
category inválida en parseo	Gemini devolvió una categoría no permitida	El prompt está bien; parsear_clasificacion debe rechazarla con ValueError

Regla práctica: si no sabes si tu código falla o es comportamiento esperado, mira si `main.py` imprime [OK] o [ERROR] con el mensaje descrito en este README.

Buenas prácticas con Git

El proyecto debe versionarse en un repositorio propio respetando este flujo de ramas:

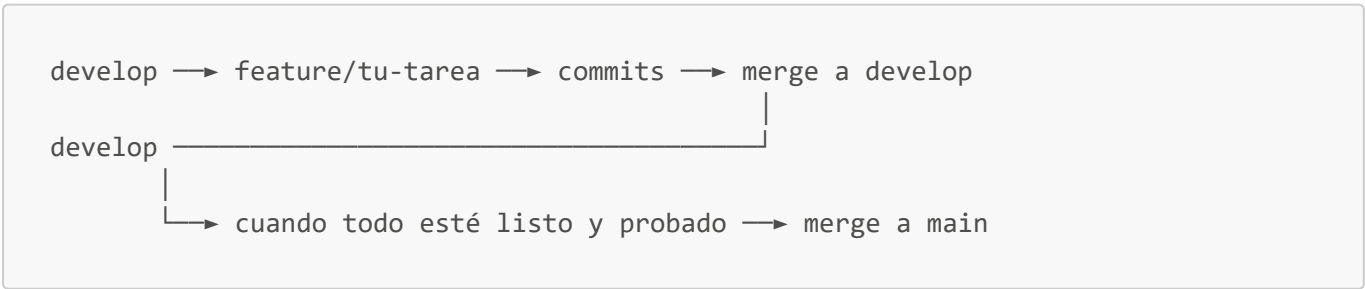
Rama	Uso
main	Código estable y entregable. No trabajes directamente aquí.
develop	Integración del trabajo de la práctica (fases 1 y 2).
Ramas secundarias	Una rama por bloque de trabajo, creada desde <code>develop</code> .

Convención de ramas secundarias

Nombres descriptivos en minúsculas, con `/`:

- `feature/validacion-y-clasificacion` — fase 1
- `feature/chat-con-contexto` — fase 2
- `fix/parseo-json` — correcciones puntuales

Flujo recomendado



Reglas mínimas

- Commits **pequeños y con mensaje claro** (qué archivo o funcionalidad tocaste).
- Nunca** subas `.env` ni `.venv/` (usa `.gitignore`).

- `develop` (o `main`) debe ejecutar `python main.py` sin errores pendientes.
-

Qué NO tienes que hacer

- Interfaz web ni menú interactivo complejo.
 - Embeddings ni vector database.
 - Fine-tuning ni seguridad avanzada.
 - Subir `.env` a Git.
-

Experimentos opcionales (si sobra tiempo)

- Guardar clasificaciones en `state.consultas_clasificadas`.
- Cambiar `WINDOW` a 2 y observar que pierde memoria.
- Añadir una entrada nueva en `data/faq.json`.